

Process documentation

Pipeline: **UCI Online Retail -> S3 -> Glue (preprocess) -> Glue (CDC simulation) -> Athena + Redshift -> Apriori -> Outliers + Similarity -> Report**. Every number below was produced by the code in this repository (see `reports/results/`); the full results are in `reports/final_report.md`.

1. Project objective

Build one connected, reproducible pipeline that extracts and cleans transactional retail data on AWS, simulates CDC, makes the data queryable and warehoused, and discovers customer purchasing patterns with association-rule mining, supported by outlier/noise detection and similarity analysis.

2. Dataset

UCI Online Retail (`Online_Retail.xlsx`, 541,909 rows x 8 columns, 2010-12-01 to 2011-12-09). AWS Glue cannot read `.xlsx`, so `data/convert_xlsx_to_csv.py` converts it to CSV (all text columns read as text so `C536379` and `85123A` are not mangled).

3. Data understanding (`data/profile_raw.py` -> `reports/results/data_profile.json`)

- Columns: InvoiceNo, StockCode, Description (text); Quantity (int); InvoiceDate (timestamp); UnitPrice (decimal); CustomerID (integer, nullable); Country (text).
- Missing: Description 1,454; CustomerID 135,080 (24.93%). Exact duplicate rows: 5,268.
- 25,900 invoices, 4,372 customers, 4,070 stock codes, 38 countries. 9,288 rows belong to cancellation invoices (`C...`), 3 to adjustment invoices (`A...`); 10,624 rows have Quantity ≤ 0 and 2,517 have UnitPrice ≤ 0 .
- Structure: one row = one product line; one invoice = one customer; (`InvoiceNo`, `StockCode`) is not unique (10,684 repeats); 650 stock codes have several description spellings; non-product codes (POST, DOT, M, BANK CHARGES, `gift_*`, ...) are mixed in.

4. Data preprocessing and 5. AWS Glue implementation (`glue/preprocess_job.py`)

Input raw CSV `--input_path`. **Output** `clean/` (Parquet), `rejected/` (Parquet + reason), `dq_report/` (JSON) under `--output_path`.

Step	Transformation	Reason
1	Read every column as text	nothing is silently converted
2	Trim, upper-case <code>StockCode</code> / <code>Description</code> , blank -> NULL; cast types (bad values become NULL, then get rejected)	consistent keys and correct types
3	<code>dropDuplicates()</code> on all 8 columns	repeated records distort counts/support
4	Reject with reason (first matching rule): <code>missing_key_field</code> , <code>cancellation_or_adjustment</code> (invoice not 6 digits), <code>invalid_quantity</code> (≤ 0), <code>invalid_price</code> (≤ 0), <code>non_product_stock_code</code> (not 5 digits + letters)	keep only real product sales; keep an audit trail
5	Add <code>line_id</code> (invoice + line number) and <code>line_total</code>	unique key for CDC; revenue
6	Data-quality report with assertions: <code>raw = duplicates + rejected + clean</code> ; no price precision lost	detects silent loss
7	Write Parquet (snappy) to the processed zone	columnar, compressed, Athena/Redshift friendly

Rows without `CustomerID` are **kept** (valid baskets); no imputation is done because a customer id cannot be invented. **Result:** 541,909 raw -> 5,268 duplicates, 14,137 rejected (9,254 cancellations/adjustments, 1,336 invalid quantity, 1,174 invalid price, 2,373 non-product codes) -> **522,504 clean rows**, 19,773 invoices, 4,334 customers, 3,791 products. A separate pandas calculation gave the same 522,504 / 19,773.

6. CDC simulation (`glue/cdc_job.py`, `cdc/inspect_cdc.py`)

A **simulation** with versioned snapshots, not a production real-time CDC system. v1 = lines up to 2011-09-30; v2 = v1 + October 2011 + synthetic changes; v3 = v2 + Nov-Dec 2011 + synthetic changes (0.5% quantity corrections, 0.2% deletes per transition, chosen by a hash of `line_id`, so fully reproducible). Changes are detected on key `line_id` with a SHA-256 row hash: I (only in new), D (only in old), U (hash differs). The change log (`changes/`, columns `op`, `change_version`) is replayed on v1 to give `current/`, and the job asserts that `current == v3`. Result: v1 357,721 -> v2 415,310 -> v3 520,962 lines; I/U/D = 58,305/1,821/716 and 106,478/2,010/826. An independent pandas replay also equals v3 (`reports/results/cdc_sample_output.txt`, with before/after examples).

7. Athena querying ([athena/](#))

`00_create_database_and_tables.sql` defines external tables over S3 Parquet (`retail_current`, `cdc_changes`, `retail_rejected`). Queries `01-06` cover totals (transactions, customers, products), revenue (month, country, hour), customers (top, frequency, average basket), products (units, revenue, support, long tail), transactions (basket size, largest invoices, product pairs) and a data-quality/CDC audit. Only portable SQL is used (explicit `CAST AS DOUBLE` before division, because Athena integer division truncates). `athena/validate_locally.py` runs every query on DuckDB over the same Parquet and writes `athena/sample_outputs/`. Athena itself was not run (no AWS account).

8. Redshift loading ([redshift/](#))

Table `retail.sales` (`DISTKEY invoice_no`, `SORTKEY invoice_date`), `COPY ... FORMAT AS PARQUET` from `s3://<bucket>/cdc/current/`, views `v_basket_items` and `v_product_names` for Apriori, and validation queries whose totals must equal Athena's. The column order of the table matches the Parquet schema (verified with pyarrow). Portable queries ran on DuckDB; DDL/COPY were not executed. Redshift is the warehouse layer: Apriori reads its baskets from it (`apriori/load_baskets.py --source redshift`).

9. Apriori implementation ([apriori/](#))

Baskets = invoices with ≥ 2 distinct products (18,267 of 19,770), items = `StockCode`. `mlxtend.apriori(low_memory=True)` on a boolean basket matrix, `min_support 0.01` (183 baskets), `min_confidence 0.5`, `min_lift 1.5`, thresholds chosen from a sensitivity table. Result: 2,474 frequent itemsets, 1,320 rules (287 single -> single). `apriori_scratch.py` is an independent implementation (tid-list bitsets) used to cross-check: identical itemsets, rules and metrics (difference 0.0). The cross-check exposed and fixed a denominator bug during development.

10. Outlier / noise detection ([notebooks/outliers_and_similarity.ipynb](#), **Part A**)

Tukey IQR on the log10 scale for quantity, unit price and line total, plus customer spend and frequency. A value is *noise* only if it is extreme (3 x IQR) **and** fully reversed by a later cancellation. 10 such lines (311,623.96, 3.05% of revenue) are excluded from value/customer analysis; all other outliers (including 38 high-spend customers = 28.98% of identified-customer revenue) are retained and treated as a VIP/wholesale segment. Details and the decision table are in the notebook.

11. Similarity / dissimilarity ([notebooks/outliers_and_similarity.ipynb](#), **Part B**)

Jaccard on customer product sets (3,698 customers with ≥ 10 products), cosine on product-basket vectors, Euclidean on standardised log-RFM. Each is validated against scipy / the textbook formula on 200 random pairs. Cosine and Apriori lift are rank-correlated (Spearman 0.870).

12. Architecture

`architecture/architecture.png` (source: `architecture/build_diagram.py`). Flow: source xlsx -> CSV -> S3 raw -> Glue job 1 -> S3 processed -> Glue job 2 -> S3 CDC zone -> Athena and Redshift -> Apriori / outlier + similarity -> report. S3 is the data lake that decouples the stages; Glue does the scalable Spark ETL; Athena gives serverless ad-hoc SQL; Redshift is the warehouse that serves mining and BI.

13-14. Results and findings

See `reports/final_report.md` sections 8-14 (all tables are generated from the result files).

15. Limitations

AWS services not executed in an AWS account; CDC changes are synthetic; cancellations excluded rather than netted; 24.9% of raw rows (25.1% of clean rows) have no customer id; Apriori at stock-code level; one retailer over one year.

16. Conclusion

All six expected activities of the assignment are implemented as one flow and every locally executable part was run and checked against an independent calculation. The AWS parts are ready to deploy with `docs/aws_deployment.md`.

Validation performed

Check	How	Result
Row reconciliation in Glue job 1	assertion <code>raw = dup + rejected + clean</code>	passed
Clean row count	independent pandas calculation	522,504 = 522,504
CDC replay	assertion in Spark + independent pandas replay	equals v3
Athena SQL	executed all 19 queries on DuckDB with same schema/data	19/19 ran
Redshift column order	pyarrow schema vs DDL	identical
Redshift totals	portable validation queries	match Athena
Apriori	from-scratch implementation	identical
Jaccard / cosine / Euclidean	scipy / formula on random pairs	identical
Pair count	Athena Q (JUMBO BAG PINK POLKADOT + RED RETROSPOT = 822 baskets) vs Apriori 0.045 x 18,267	consistent

How to run

`bash run_local_pipeline.sh Online_Retail.xlsx` (needs Java 17+ for PySpark; see `README.md`). On AWS follow `docs/aws_deployment.md`.